Language: **English** Italiano Українська

h4nb guide

Direct N-body on NVIDIA GPUs: 4th-order Hermite, block time steps, FP64

> ✅ = works on `main` now. 🚧 = agreed design, not built yet (task number in brackets).

Contents

1. The problem
2. The approach
3. Architecture
4. Quick start
5. Configuration
6. Cheat sheet
7. Common problems
8. Hardware
9. Documents

1. The problem

A researcher studies a disk or a cluster around one heavy body: gas clouds around an active galactic nucleus (AGN), or stars around a central mass. The particles can have 10 % to 100 % of the central mass, so the central body moves.

- **Exact forces:** every particle pulls on every other particle. One full force pass at N = 1 024 000 is about 10^{12} pairs.
- **Controlled accuracy:** the maximum relative energy error over one mean orbital period is 1e-8 or less. A measured run must show it.
- **Speed:** large runs finish in hours on data-centre GPUs.
- **Reproducible:** same input, configuration, devices and build give the same output bit for bit.

2. The approach

Topic	Decision	Why
Integrator	4th-order Hermite: predict, compute a and jerk, correct	4th order with one force pass per step
Time steps	Aarseth criterion (η), power-of-two block steps, integer ticks	Only active particles get a new force; exact block times
Central body	Particle 0, full participant	Keeps momentum conserved
Precision	FP64 only, no fast-math, no float atomics, fixed sum order	Accuracy and bit-identical runs
Softening	Optional Plummer ε; presets AGN 0.01, stellar 0.0001	It changes the model; it does not hide errors

Topic	Decision	Why
Several GPUs	Split sources (j-parallel); owner adds partials in order	Few active particles per step; same work per GPU
Several nodes 🚧	One MPI rank per GPU, CUDA-aware MPI (T029–T036)	Target: 4–8 GH200
η	Measure it per system; about 5e-4 for 1e-8	The TZ range 0.001–0.02 does not reach 1e-8
Speed work	Each optimisation needs a measured gate	GPU speed guesses are often wrong

3. Architecture

One public header (`include/h4nb/h4nb.hpp`) and one class, `Simulation`. No CUDA type in the API. The GPU kernels and the CPU test oracle share one math header (`src/hermite.cuh`).

One block step:

1. Find the next block time Tn = min(next).
2. Predict all particles to Tn.
3. Select the active particles (next == Tn).
4. Send the active targets to every GPU.
5. Each GPU computes the partial a and jerk of its own sources on all targets (shared-memory tiles, one thread per target).
6. Send each slice to the owner GPU.
7. The owner adds the partials in device order, corrects, and chooses the new step.

Memory: about 0.4 GB per GPU at 1M particles. Compute is the limit.

4. Quick start

Build (Linux x86_64 or aarch64, CUDA ≥ 12.0, CMake ≥ 3.24, GPU compute capability ≥ 7.0):

```
make build                                    # Release, CMAKE_CUDA_ARCHITECTURES=native
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES=89      # RTX 40
70 Ti
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES=90      # GH200
/ H100
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES="89;90" # both
cmake --build build -j
```

Run:

```
build/h4nb_mkdisk n=127999 frac=0.1 out=disk128k.snap     # prints mean / median / mass-we
ighted period
build/h4nb_run ic=disk128k.snap out=run128k t_end=6.3125 diag=0.0625 eta=0.0005 devices=0,
1
build/h4nb_order e=0.5                                    # Kepler orbit + 4th-order chec
k
build/h4nb_bench n=16384,131072,1024000 devices=0,1       # speed table
DT_MAX="0.015625 0.0078125" tools/eta_scan.sh disk128k.snap 6.3125 0.0625 0,1   # eta stud
y
```

`t_end`, `diag` and `snap` must be multiples of `dt_max` (default 1/64), and `diag` must divide `t_end`.

5. Configuration

✅ Keys of `h4nb_run` now (command line `key=value`):

Key	Default	Meaning
`ic`	—	initial conditions (text or .snap)
`out`	—	output directory
`t_end`	—	end time
`eta`	`0.01`	Aarseth accuracy (use ≈0.0005 for 1e-8)
`eta_init`	`0.01`	first step factor
`dt_max`	`1/64`	largest step (level 0)
`max_level`	`40`	dt_min = dt_max / 2^max_level
`eps`	`0`	softening length ε
`mode`	`block`	block or fixed
`fixed_level`	`0`	level in fixed mode
`fixed_center`	`0`	1: centre does not move (tests)
`devices`	`0`	GPU list, e.g. 0,1
`diag`	`dt_max`	energy record interval
`snap`	`0`	snapshot interval (multiple of diag)
`r_close`	`auto`	close-pair radius; 0 = off

🚧 Planned (T004–T013): one key table, five sources. A later source wins:

1. default
2. `preset=none|agn|stellar`
3. `config=<file>` (key=value lines)
4. environment `H4NB_<KEY>`
5. command line

An unknown key or a bad value is an error (exit code 2). Each run writes `config.used`. `help=1` prints all keys.

Planned keys	Task
`n, central_id, period, period_kind`	T011
`preset, G, eps2`	T006, T011
`dissipation, drag_tau, drag_vbg`	T041–T043
`tile, blocks_per_sm`	T008, T017–T019
`device_offset, mpi_host_staging`	T030, T032
`snap_times, energy_tol, perf`	T011, T014
`sizes, tiles, bpsm, fp64_peak`	T019
`etas, dt_maxes, t_ends`	T015

How to choose the main values

`eta`	Measure it with the η study. Start near 0.0005. Cost grows as 1/√η.
`dt_max`	1/64 to 1/16 of the shortest period of interest.
`max_level`	Raise it if you see the warning dt < dt_min.
`eps`	0 for point masses; preset=agn or preset=stellar for the two model types.
`diag`	One energy check costs $N^2/2$ pairs. At 1M use T/16 or more.
`tile, blocks_per_sm` 🚧	Measure with h4nb_bench; keep the best values in the machine config.
`GPUs / nodes`	Set by the launcher (devices=, mpirun, srun). The cluster size is not fixed in the code.

6. Cheat sheet

Config file 🚧

```
# configs/agn.cfg   (planned: T001, T011)
preset=agn                    # eps = 0.01
eta=0.0005                    # measured value for 1e-8
dt_max=0.015625
diag=0.0625
period=6.283185307179586      # 2*pi for G = 1, M = 1, a = 1
```

Run with a config file 🚧

```
build/h4nb_run config=configs/agn.cfg ic=disk128k.snap out=run1 t_end=6.3125 devices=0,1
H4NB_ETA=0.00025 build/h4nb_run config=configs/agn.cfg ic=disk128k.snap out=run2 t_end=6.3
125
build/h4nb_run config=run1/config.used out=run1_repeat     # repeat a run exactly
```

Several nodes 🚧

```
srun --ntasks-per-node=4 --gpus-per-task=1 --gpu-bind=single:1 \
  build/h4nb_run config=configs/gh200.cfg ic=disk1m.snap out=run1m t_end=6.3125
mpirun -np 8 --map-by ppr:4:node --bind-to core \
  build/h4nb_run config=configs/gh200.cfg ic=disk1m.snap out=run1m t_end=6.3125
```

C++ ✅

```
#include <cmath>
#include <h4nb/h4nb.hpp>

h4nb::Particles p = h4nb::read_snapshot("disk128k.snap");   // or h4nb::load_text("ic.tx
t")
h4nb::Config c;
c.eta = 5e-4;  c.dt_max = 1.0 / 64;  c.eps2 = 0.01 * 0.01;  c.devices = {0, 1};
h4nb::Simulation sim(p, c);
const auto e0 = sim.diagnostics();
sim.advance_to(6.3125);                                      // a multiple of dt_max
const auto e1 = sim.diagnostics();
double rel = std::abs((e1.E - e0.E) / e0.E);                 // the 1e-8 goal
if (sim.status() & h4nb::kNaN) { /* stop */ }
```

Output files ✅

`energy.dat`	`time K U E rel_err max_rel_err mean_rel_err`
`momentum.dat`	`time Px Py Pz Lx Ly Lz dP_rel dL_rel`
`timesteps.dat`	`time dt_min dt_max dt_mean mean_active r_min`
`performance.dat`	`time wall_s block_steps active pairs_per_s`
`snap_NNNNNN.snap`	`binary: header + id m x y z vx vy vz`

7. Common problems

Symptom	Cause	Action
max |dE/E| > 1e-8	η too large	Lower eta (try 0.0005, then 0.00025). Run the η study.
warning: dt < dt_min	A particle needs a finer step	Raise max_level. Check close pairs.
warning: pair closer than r_close	Close encounter	Check r_min in timesteps.dat. Consider softening if the model allows it.
non-finite value (NaN or Inf)	Bad input or a collision	Check the input. The run stops by design.
advance_to: t must be a multiple of dt_max	t_end, diag or snap not on the grid	Use multiples of dt_max (1/64 = 0.015625).
Build fails with arch native	No GPU on the build machine	Set CMAKE_CUDA_ARCHITECTURES=89 or 90.
A key has no effect	Unknown keys are ignored today	Check the spelling. 🚧 T005 makes it an error.
1M run is very slow on RTX	Low FP64 rate (1/64)	Use RTX for N ≤ 131 072; use GH200 for 1M.
Results differ between GPU counts	Different sum order	Expected. Compare with the written tolerance, not bit for bit.

8. Hardware

GPU	Use	Note
RTX 4070 Ti (8.9)	Development, N ≤ 131 072	≈17 h per 1M period (estimate)
GH200 (9.0)	1M and production runs	≈18 min per 1M period on one GPU (estimate)
Below 7.0 (GTX 1060)	Not supported	Constitution 2.0.0

9. Documents

- README — `README.md`
- Constitution — `.specify/memory/constitution.md`
- Spec — `specs/001-hermite-nbody-lib/spec.md`
- Plan — `specs/001-hermite-nbody-lib/plan.md`
- Research — `specs/001-hermite-nbody-lib/research.md`
- Run configuration contract — `specs/001-hermite-nbody-lib/contracts/run-config.md`
- Tasks — `specs/001-hermite-nbody-lib/tasks.md`
- Quickstart — `specs/001-hermite-nbody-lib/quickstart.md`

- Multi-node design — `docs/multi-node-design.md`
- CUDA best practices — `docs/cuda-best-practices.md`

Generated from the repository documents by docs/guide/build_guide.py. build_guide.py